

# What Happen When You Press Power Button On Computer

## Documentation

### Assignmenet- I

## Actions Initiated by Pressing the Power Button on a Computer

- When you push the power button on a computer, it triggers a complex series of events. Initial Signal Transmission starts with an electrical signal transmitted to the **PSU or battery**, which then initiates Power Supply Activation, and power begins to flow to essential components such as the motherboard and CPU. Next is **BIOS/UEFI and Bootloader Engagement**, where the firmware initializes hardware and triggers the bootloader. The system performs a diagnostic check on the hardware components in POST, ensuring that everything is working as expected.
- Once these are successful, **OS Loading begins** the bootloader loads the **OS into memory**, after which comes kernel initialization, dealing with system resource management and communication with hardware. Then, loads the **Device Drivers**, by which the operating system can access the hardware periphery. Lastly, it initializes the Startup Programs and Services; thus, services like networking, security, etc., are turned on.
- Finally, the UI Presentation is presented, and the user can interact with the system. In shutdown or sleep modes, the Shutdown and Sleep Modes processes are performed. The OS saves data, closes applications, and either safely powers down the hardware or conserves energy while saving the current state to RAM for quick resumption.

## Some Points Of Step By Step:

- Initial Signal Transmission
- Power Supply Activation
- BIOS/UEFI and Bootloader Engagement
- POST (Power-On Self-Test)
- Operating System Loading
- Kernel Initialization
- Device Drivers Loading
- Startup Programs and Services
- User Interface (UI) Presentation

## 1. Initial Signal Transmission

When we press the power button, the electrical signal is sent to **power supply unit (PSU)** or **device's battery** to the motherboard. Startup process by activating key components like the CPU and memory, allowing them to begin their initial operations and eventually boot the operating system.

### Desktop computers

The power button is connected to the PSU via a small cable. When the button is pressed, it closes a circuit, allowing a small amount of current to flow and signal the PSU to start providing power to the motherboard and other components. This initial burst of power is what gets the whole system moving.

### Laptops and mobile devices

The power button similarly sends a signal to the built-in battery, instructing it to begin supplying power to the device's components. The initial signal transmission is a critical step in the power-up process, as it initiates the sequence of events that lead to the booting of the operating system and the eventual ready state of the device.

**Ex.** Power On Signal, CMOS, Power Sequence

## 2. Power Supply Activation

Once the initial signal is transmitted, the PSU or battery begins to supply power to the device's components. In desktop computers, the PSU converts AC power from the wall outlet into the DC power required by the internal components. In laptops and mobile devices, the battery provides the necessary power. This step ensures that the motherboard, **CPU, RAM**, and other critical hardware receive the power they need to start up and function properly.

### In Desktop Computers:

The **power supply unit (PSU)** converts **alternating current (AC)** from the wall outlet into the **direct current (DC)** needed by the computer's internal components. When the power button is pressed, it completes a circuit that allows a small current to flow, waking the PSU from standby mode. The PSU then begins supplying power to the **motherboard, CPU, RAM, storage devices**, and other peripherals. Modern PSUs have multiple voltage rails, each providing specific voltage levels (e.g., **3.3V, 5V, 12V**) to different components.

### In Laptops and Mobile Devices:

Laptops and mobile devices rely on an internal battery as their primary power source. When the power button is pressed, it signals the battery management system to regulate and distribute power. This system ensures that each component receives the correct voltage and current. Unlike desktops, laptops and mobile devices feature more integrated and energy-efficient power delivery systems, designed to optimize energy consumption and extend battery life.

### Components Powered:

- **Motherboard:** Acts as the central hub connecting all other components.
- **CPU (Central Processing Unit):** The brain of the computer, responsible for executing instructions.
- **RAM (Random Access Memory):** Temporary storage for data being actively used or processed by the CPU.
- **Storage Devices:** Hard drives or SSDs that store the operating system, applications, and user data.
- **Peripherals:** Devices such as keyboards, mice, displays, and network cards.

### 3. BIOS/UEFI and Bootloader Engagement

#### BIOS/UEFI Initialization:

BIOS (**Basic Input/Output System**) and UEFI (**Unified Extensible Firmware Interface**) are firmware interfaces responsible for initializing and testing hardware components during system startup. When a computer is powered on, the BIOS or UEFI firmware is the first software to execute, residing in a non-volatile memory chip on the motherboard.

BIOS, the older of the two, operates in **16-bit mode** with a limitation of 1 MB addressable space and follows a sequential boot process. UEFI, its modern successor, supports **32-bit and 64-bit** modes, allows for larger storage device compatibility, and offers an enhanced graphical interface. Additionally, **UEFI includes Secure Boot**, a security feature that ensures only trusted software is loaded during startup, preventing unauthorized code execution.

#### Hardware Initialization

- **CPU Initialization** – Ensures the processor is functional and ready to execute instructions.
- **Memory Check (RAM)** – Tests and configures system memory for use.
- **Storage Controller Setup** – Detects and configures hard drives, SSDs, and other storage devices.
- **Peripheral Initialization** – Prepares essential devices such as keyboards, mice, and network adapters.

#### Boot Device Selection

- **Internal storage** (HDDs or SSDs)
- **External media** (USB drives, optical disks)
- **Network boot options** (PXE boot)

#### Bootloader Activation

- Loading the operating system kernel into memory
- Initializing essential system components
- Transitioning control to the **OS**
- Bootloader Execution
- The bootloader plays a pivotal role in the system startup process.

**Windows Boot Manager – Handles the boot process for Windows-based systems.**  
**GRUB (Grand Unified Bootloader)**

A widely used bootloader for Linux distributions. If multiple operating systems or configurations are available, the bootloader may present a selection menu, allowing the user to choose the desired OS. Once an option is selected, the bootloader proceeds to load the operating system kernel and complete the startup sequence.

#### Significance of BIOS/UEFI and Bootloader

**BIOS/UEFI** initialization is essential for ensuring that all hardware components are correctly configured before the system boots. Without it, the computer would be unable to function. The bootloader further bridges the gap between hardware and software, ensuring that the operating system is loaded correctly and ready for use.

## **4.Power-On Self-Test (POST)**

**Power-On Self-Test (POST)** is an essential diagnostic check that runs as soon as a computer is powered on. This process ensures that all hardware components are functioning properly before the operating system loads.

### **What Happens During POST?**

#### **1. Initial Checks**

Once BIOS/UEFI firmware initializes, the POST process begins. The system first checks whether the **CPU** is operational and verifies that the required voltage levels are stable.

#### **2. Memory Testing**

The system tests the **RAM (Random Access Memory)** to ensure that the installed memory modules are working correctly. If there's an issue with the RAM, the POST process may fail, often signaling the problem with **beep codes** or an error message.

#### **3. Peripheral Device Initialization**

The system then checks basic input devices like the **keyboard and mouse** to confirm they are properly connected and functioning.

#### **4. Storage Device Detection**

The firmware detects and verifies **storage devices** such as hard drives and SSDs, ensuring they are accessible and contain the necessary boot files.

#### **5. Graphics Card Initialization**

The graphics card is initialized, allowing basic video output. This is why you see the manufacturer's logo or firmware screen when the computer starts up.

#### **6. Additional Hardware Checks**

Depending on the system, POST may also check other components like **network cards, sound cards, and additional peripherals** to ensure everything is working correctly.

### **Handling Errors During POST**

**POST** detects an issue, it will halt the boot process and provide feedback in one or more ways:

- **Beep Codes:** A series of beeps from the system speaker, with different patterns indicating specific issues.
- **Error Messages:** Displayed on-screen to explain the problem.
- **LED Indicators:** Some motherboards have built-in diagnostic LEDs to help pinpoint faulty components.

### **Why Is POST Important?**

POST plays a crucial role in ensuring the system is stable and free from hardware issues before attempting to load the operating system. By detecting failures early, it helps prevent crashes, instability, and potential data loss.

### **What Happens Next?**

If POST completes successfully without detecting any issues, the BIOS/UEFI moves on to the next stage: **locating and loading the bootloader**, which then starts the operating system.

## 5. Operating System Loading

The process of loading the operating system (**OS**) is a critical phase in the boot sequence, during which the core components of the OS are transferred from storage into the system's memory (**RAM**). This step allows the OS to take control of the hardware and provide a fully functional environment for the user.

### Bootloader Responsibility

Once BIOS/UEFI completes initialization and POST, the **bootloader** takes over. The bootloader is a small program stored in a designated section of the storage device, known as the **boot sector**.

- Locating the OS **kernel** on the storage device.
- Loading the kernel into memory.
- Handing control over to the operating system.

### Loading the Kernel

**kernel** is the core of the operating system, responsible for managing hardware resources and facilitating communication between hardware and software. The bootloader reads the kernel files from storage and transfers them into RAM, making them ready for execution.

## 6. Kernel Initialization

- Configuring **memory management**.
- Initializing **device drivers** to enable hardware communication.
- Mounting the **root file system**, which provides access to essential OS files and resources.

### Starting Essential System Processes

- In Linux and other Unix-based systems, the first process started is **init** (or **systemd** in modern distributions), which is responsible for managing system services.
- In Windows, similar background processes and system services are initialized to ensure smooth operation.

### Graphical User Interface (GUI) Initialization

If the system is configured with a **graphical user interface (GUI)**, the OS loads the necessary graphical drivers, configures display settings, and starts the desktop environment or window manager.

- On systems without a GUI, a **command-line interface (CLI)** is presented instead.

### User Authentication and Login

Once the system is ready, a **login screen** appears, prompting users to enter their credentials. This authentication step ensures that only authorized users can access system resources.

### Finalizing the Boot Process

After a successful login, the OS completes its loading process by:

- Starting user-specific processes and applications.
- Loading personalized settings and preferences.
- Displaying the desktop environment, allowing the user to interact with the system.

### Examples of OS Loading in Different Systems

- In **Windows**, the **Windows Boot Manager** loads the kernel (**ntoskrnl.exe**) and initiates the OS startup process.
- In **Linux**, the **GRUB bootloader** loads the Linux kernel (**vmlinuz**) and begins system initialization.

## 7. Device Drivers Loading

**Device drivers loading phase** is essential for enabling communication between the operating system and hardware components. This process ensures that peripherals such as graphics cards, printers, and network adapters function correctly.

### **What Are Device Drivers?**

Device drivers are specialized software programs that act as **bridges** between the operating system and hardware. They allow the OS to send commands to hardware components and receive feedback for seamless operation.

### **How Are Device Drivers Loaded?**

- During startup, the OS scans the system for **connected hardware** and loads the appropriate drivers into **RAM**.
- Drivers are typically stored on the system's **storage device** and retrieved as needed.
- Some critical drivers, such as storage and keyboard drivers, are preloaded during early boot stages, while others load later.

### **Why Are Device Drivers Important?**

Without proper drivers, the OS **cannot interact** with hardware components, leading to malfunctions or limited functionality.

### **Examples of Essential Device Drivers:**

- **Graphics Drivers**: Enable smooth rendering of images, videos, and animations.
- **Network Drivers**: Provide connectivity to Wi-Fi, Ethernet, and other network interfaces.
- **Audio Drivers**: Allow sound playback and microphone input.

## **8. Startup Programs and Services**

### **System Services:**

- Networking (**Wi-Fi, Bluetooth, LAN**)
- Security (**Antivirus, Firewall, Encryption**)
- System Management (**Power settings, Performance monitoring**)

### **User Startup Programs:**

- Antivirus software for real-time protection.
- Cloud storage clients like Google Drive or OneDrive.
- Productivity tools such as messaging apps or task managers.

### **Why Are Startup Programs & Services Important?**

- System services ensure core functionalities (**network, security, performance**) are available immediately.
- Startup programs improve convenience by making frequently used applications instantly accessible.

## **9. User Interface (UI) Presentation**

- The final phase of the startup process is the User Interface **(UI)** presentation
- OS provides an interactive environment for users.

Graphical User Interface **(GUI)** Initialization:

- The OS loads graphical drivers and applies display settings.
- The window manager or desktop environment starts, defining how applications and menus appear.

Login Screen & Authentication:

- Users are presented with a login screen to authenticate their credentials.
- This step ensures security by restricting unauthorized access.

Desktop Environment & Customization:

After login desktop environment fully loads, allowing users to:

- Access applications, manage files, and configure settings.
- Customize the interface with wallpapers, themes, icons, and shortcuts.

### **Why Is UI Presentation Important?**

The UI is the primary way users interact with the OS. A well-designed UI enhances:

- Usability: Easy navigation and intuitive controls.
- Efficiency: Quick access to apps and system settings.

## **Conclusion:**

Powering on a computer triggers a coordinated sequence of events, transitioning it from an inactive state to a fully operational system. It begins with **power activation**, where the PSU or battery supplies energy to essential components. **BIOS/UEFI initialization** follows, conducting hardware diagnostics through **POST** and engaging the **bootloader** to load the operating system. Once the **OS kernel** is initialized, **device drivers** ensure seamless communication between hardware and software, while **startup programs and services** enhance functionality. Finally, the **user interface** is presented, allowing interaction with the system. This structured process ensures efficiency, stability, and a smooth user experience.